```
import numpy as np
import random

# ---------------------------------------
# 1. DEFINE THE CLOUD RESOURCE PROBLEM
# ---------------------------------------

tasks = [
    {"cpu": 4, "memory": 4},
    {"cpu": 6, "memory": 3},
    {"cpu": 2, "memory": 2},
    {"cpu": 5, "memory": 4},
    {"cpu": 3, "memory": 2},
    {"cpu": 4, "memory": 3}
]

vms = [
    {"cpu": 10, "memory": 10, "speed": 5, "power": 50},
    {"cpu": 12, "memory": 8, "speed": 4, "power": 60},
    {"cpu": 8, "memory": 8, "speed": 3, "power": 40}
]

NUM_TASKS = len(tasks)
NUM_VMS = len(vms)


# ---------------------------------------
# 2. INITIALIZE GWO PARAMETERS
# ---------------------------------------

N = 10
MAX_ITER = 100


# ---------------------------------------
# 3. INITIALIZE WOLF POPULATION
# ---------------------------------------

wolves = np.random.uniform(
    0, NUM_VMS - 1, (N, NUM_TASKS)
)


# Convert continuous positions into VM assignments
def decode(position):

    return [
        min(NUM_VMS - 1, max(0, int(round(x))))
        for x in position
    ]


# ---------------------------------------
# 4. EVALUATE FITNESS
# ---------------------------------------

def fitness(position):

    allocation = decode(position)

    cpu_used = [0] * NUM_VMS
    memory_used = [0] * NUM_VMS
    vm_time = [0] * NUM_VMS
    energy = 0

    # Check resource constraints
    for i, vm_id in enumerate(allocation):

        task = tasks[i]
        vm = vms[vm_id]

        cpu_used[vm_id] += task["cpu"]
        memory_used[vm_id] += task["memory"]

    # Calculate resource violation penalty
```

```
    penalty = 0

    for j in range(NUM_VMS):

        cpu_over = max(
            0, cpu_used[j] - vms[j]["cpu"]
        )

        memory_over = max(
            0, memory_used[j] - vms[j]["memory"]
        )

        penalty += cpu_over + memory_over

    # Calculate execution time and energy
    for i, vm_id in enumerate(allocation):

        task = tasks[i]
        vm = vms[vm_id]

        execution_time = task["cpu"] / vm["speed"]

        vm_time[vm_id] += execution_time

        energy += execution_time * vm["power"]

    makespan = max(vm_time)

    # Combined objective function
    max_possible_time = sum(
        task["cpu"] / min(vm["speed"] for vm in vms)
        for task in tasks
    )

    max_possible_energy = sum(
        (task["cpu"] / min(vm["speed"] for vm in vms))
        * max(vm["power"] for vm in vms)
        for task in tasks
    )

    score = (
        0.5 * makespan / max_possible_time
        + 0.5 * energy / max_possible_energy
        + 100 * penalty
    )

    return score


# ----------------------------------------
# 5. UPDATE WOLF POSITIONS
# ----------------------------------------

def update_position(wolf, alpha, beta, delta, a):

    A1 = 2 * a * random.random() - a
    A2 = 2 * a * random.random() - a
    A3 = 2 * a * random.random() - a

    C1 = 2 * random.random()
    C2 = 2 * random.random()
    C3 = 2 * random.random()

    D_alpha = abs(C1 * alpha - wolf)
    D_beta = abs(C2 * beta - wolf)
    D_delta = abs(C3 * delta - wolf)

    X1 = alpha - A1 * D_alpha
    X2 = beta - A2 * D_beta
    X3 = delta - A3 * D_delta

    new_position = (X1 + X2 + X3) / 3

    # Keep positions within valid VM range
    return np.clip(new_position, 0, NUM_VMS - 1)


# ----------------------------------------
```

```
# 6. ITERATE GWO
# ---------------------------------------

alpha = None
beta = None
delta = None

best_score = float("inf")

for iteration in range(MAX_ITER):

    scores = [fitness(wolf) for wolf in wolves]

    # Identify Alpha, Beta and Delta wolves
    sorted_indices = np.argsort(scores)

    alpha = wolves[sorted_indices[0]].copy()
    beta = wolves[sorted_indices[1]].copy()
    delta = wolves[sorted_indices[2]].copy()

    current_best = scores[sorted_indices[0]]

    if current_best < best_score:
        best_score = current_best
        best_position = alpha.copy()

    # Decrease a from 2 to 0
    a = 2 - (2 * iteration / MAX_ITER)

    # Update all wolf positions
    for i in range(N):

        wolves[i] = update_position(
            wolves[i], alpha, beta, delta, a
        )

    if (iteration + 1) % 10 == 0:
        print(
            "Iteration:", iteration + 1,
            "Best Fitness:", round(best_score, 4)
        )


# ---------------------------------------
# 7. OUTPUT THE BEST SOLUTION
# ---------------------------------------

best_allocation = decode(best_position)

print("\n------ BEST CLOUD RESOURCE ALLOCATION ------")

for i, vm_id in enumerate(best_allocation):

    print(
        "Task", i + 1,
        "-> VM", vm_id + 1
    )

print("\nBest Fitness:", round(best_score, 4))

# Calculate final resource usage
print("\n------ VM RESOURCE UTILIZATION ------")

for j in range(NUM_VMS):

    assigned_tasks = [
        i + 1 for i, vm_id in enumerate(best_allocation)
        if vm_id == j
    ]

    cpu = sum(
        tasks[i - 1]["cpu"] for i in assigned_tasks
    )

    memory = sum(
        tasks[i - 1]["memory"] for i in assigned_tasks
    )
```

```
    print("VM", j + 1)
    print("Assigned Tasks:", assigned_tasks)
    print("CPU Used:", cpu, "/", vms[j]["cpu"])
    print("Memory Used:", memory, "/", vms[j]["memory"])

# Calculate final makespan and energy
final_score = fitness(best_position)

vm_times = [0] * NUM_VMS
total_energy = 0

for i, vm_id in enumerate(best_allocation):

    execution_time = tasks[i]["cpu"] / vms[vm_id]["speed"]

    vm_times[vm_id] += execution_time

    total_energy += execution_time * vms[vm_id]["power"]

print("\nMakespan:", round(max(vm_times), 2))
print("Energy Consumption:", round(total_energy, 2))
```

```
Iteration: 10 Best Fitness: 0.4861
Iteration: 20 Best Fitness: 0.4861
Iteration: 30 Best Fitness: 0.4861
Iteration: 40 Best Fitness: 0.4861
Iteration: 50 Best Fitness: 0.4861
Iteration: 60 Best Fitness: 0.4861
Iteration: 70 Best Fitness: 0.4861
Iteration: 80 Best Fitness: 0.4861
Iteration: 90 Best Fitness: 0.4861
Iteration: 100 Best Fitness: 0.4861

------ BEST CLOUD RESOURCE ALLOCATION ------
Task 1 -> VM 3
Task 2 -> VM 1
Task 3 -> VM 1
Task 4 -> VM 2
Task 5 -> VM 2
Task 6 -> VM 3

Best Fitness: 0.4861

------ VM RESOURCE UTILIZATION ------
VM 1
Assigned Tasks: [2, 3]
CPU Used: 8 / 10
Memory Used: 5 / 10
VM 2
Assigned Tasks: [4, 5]
CPU Used: 8 / 12
Memory Used: 6 / 8
VM 3
Assigned Tasks: [1, 6]
CPU Used: 8 / 8
Memory Used: 7 / 8

Makespan: 2.67
Energy Consumption: 306.67
```